

LAPORAN PRAKTEK
Pemrograman Struktur Data
(Basis Data NoSQL)



DHIA ATHALLA (225443006)



TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI
Politeknik Manufaktur Bandung
Tahun Ajaran 2025/2026

PENDAHULUAN

Dalam praktikum ini, kita menggunakan konsep *aggregation pipeline*. Konsep *Aggregation framework* adalah alat utama di MongoDB untuk memproses, mengubah, dan menganalisis data. Data mengalir melalui serangkaian *stage*, di mana output dari satu stage menjadi input stage berikutnya. Tujuan mempelajari serangkaian materi ini agar dapat menjelaskan konsep aggregation pipeline dan cara kerjanya serta mampu menggunakan stage-stage utama: `$match`, `$group`, `$sort`, `$project`, `$unwind`, `$lookup`, `$addFields`, `$bucket`, `$facet` dan mampu menerapkan akumulator (`$sum`, `$avg`, `$min`, `$max`, `$push`, `$addToSet`, `$first`, `$last`) untuk menghasilkan ringkasan data industri.

Selain itu, kita juga bisa memanfaatkan operator ekspresi (`$dateToString`, `$hour`, `$cond`, `$ifNull`, `$multiply`, `$divide`) untuk transformasi data di dalam pipeline. Serta menggabungkan data dari beberapa koleksi menggunakan `$lookup` (join) dan merancang pipeline agregasi untuk menjawab pertanyaan bisnis nyata: total produksi per mesin, rata-rata suhu per jam, reject rate, OEE, distribusi nilai sensor. Dengan hal itu kita bisa mengoptimasi pipeline dengan menempatkan `$match` dan `$limit` di awal, serta memproyeksikan field yang diperlukan saja.

DESAIN DATA

TUTORIAL TERBIMBING STUDI KASUS AGREGASI

data_sensor	inspeksi
_id: objectId - category: string - nilai: double - sensor_id: string - timestamp: date	_id: objectId - batch_id: string - cacat_ditemuk...: int - inspektor: string - jenis_cacat: [] - jumlah_diperi...: int - mesin: string - shift: int - tanggal: date
sensor_info	target_kualitas
_id: objectId - nama_sensor: string - sensor_id: string	_id: objectId - batch_id: string - target_maks_c...: int

PANDUAN TUGAS MANDIRI PERTEMUAN 5

produksi_harian
_id: objectId - actual_ok: int - actual_reject: int - durasi_operas...: int - durasi_tersed...: int - mesin: string - shift: int - tanggal: date - target: int

PIPELINE DAN HASIL

1. Kode Pipeline

a. TUTORIAL TERBIMBING STUDI KASUS AGREGASI

- Trend Bulanan

```
db.produksi_harian.aggregate([
  // Tahap 1: Tidak perlu $match karena ingin semua data
  // Tahap 2: Group by mesin dan bulan
  { $group: {
    _id: {
      mesin: "$mesin",
      bulan: { $month: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  // Tahap 3: Hitung OEE
  { $project: {
    _id: 0,
    mesin: "$_id.mesin",
    bulan: "$_id.bulan",
    tahun: "$_id.tahun",
    total_ok: 1,
    total_target: 1,
    total_reject: 1,
    availability: { $divide: ["$total_op", "$total_avail"] },
    performance: { $divide: ["$total_ok", "$total_target"] },
    quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target"] },
        { $divide: ["$total_op", "$total_avail"] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
      ]
    }
  }},
  // Tahap 4: Urutkan berdasarkan OEE terendah
  { $sort: { OEE: 1 } }
])
```

- **Mesin Terburuk**

```
db.produksi_harian.aggregate([
  // Tahap 1: Tidak perlu $match karena ingin semua data
  // Tahap 2: Group by mesin dan bulan
  { $match: {
    tanggal: {
      $gte: new Date("2026-06-01"),
      $lte: new Date("2026-06-30T23:59:59")
    }
  }},
  { $group: {
    _id: {
      mesin: "$mesin"
    },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  // Tahap 3: Hitung OEE
  { $project: {
    _id: 0,
    mesin: "$_id.mesin",
    bulan: "$_id.bulan",
    total_ok: 1,
    total_target: 1,
    total_reject: 1,
    availability: { $divide: ["$total_op", "$total_avail"] },
    performance: { $divide: ["$total_ok", "$total_target"] },
    quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target"] },
        { $divide: ["$total_op", "$total_avail"] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
      ]
    }
  }},
  // Tahap 4: Urutkan berdasarkan OEE terendah
  { $sort: { OEE: 1 } }
])
```

- **Kinerja Harian**

```

db.produksi_harian.aggregate([
  // Tahap 1: Tidak perlu $match karena ingin semua data
  // Tahap 2: Group by mesin dan bulan

  { $group: {
    _id: {
      mesin: "$mesin",
      hari: { $dateToString: { format: "%Y-%m-%d", date: "$tanggal" } }
    },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }},
  // Tahap 3: Hitung OEE
  { $project: {
    _id: 0,
    mesin: "$_id.mesin",
    bulan: "$_id.bulan",
    total_ok: 1,
    total_target: 1,
    total_reject: 1,
    availability: { $divide: ["$total_op", "$total_avail"] },
    performance: { $divide: ["$total_ok", "$total_target"] },
    quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
    OEE: {
      $multiply: [
        { $divide: ["$total_ok", "$total_target"] },
        { $divide: ["$total_op", "$total_avail"] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
      ]
    }
  }},
  // Tahap 4: Urutkan berdasarkan OEE terendah
  { $sort: { hari: -1 , mesin : 1 } }
])

```

- **Simulasi Perbaikan**

```

db.produksi_harian.aggregate([
  // Tahap 1: Tidak perlu $match karena ingin semua data
  // Tahap 2: Group by mesin dan bulan

  { $group: {
    _id: {
      mesin: "$mesin",

```

```

    hari: { $dateToString: { format: "%Y-%m-%d", date: "$tanggal" } }
  },
  total_target: { $sum: "$target" },
  total_ok: { $sum: "$actual_ok" },
  total_reject: { $sum: "$actual_reject" },
  total_op: { $sum: "$durasi_operasi_menit" },
  total_avail: { $sum: "$durasi_tersedia_menit" }
}},
// Tahap 3: Hitung OEE
{ $project: {
  _id: 0,
  mesin: "$_id.mesin",
  bulan: "$_id.bulan",
  total_ok: 1,
  total_target: 1,
  total_reject: 1,
  availability: { $divide: ["$total_op", "$total_avail"] },
  performance: { $divide: ["$total_ok", "$total_target"] },
  quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
  OEE: {
    $multiply: [
      { $divide: ["$total_ok", "$total_target"] },
      { $divide: ["$total_op", "$total_avail"] },
      { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
    ]
  }
}},
{ $match: {
  quality: { $lt: 0.95 }
}},
// Tahap 4: Urutkan berdasarkan OEE terendah
{ $sort: { quality : 1 } }
])

```

b. PANDUAN TUGAS MANDIRI PERTEMUAN 5

- Menampilkan rata rata nilai per sensor_id pada 7 hari terakhir.

```

db.data_sensor.aggregate([
  // Tahap 1: Filter 7 hari terakhir dari sekarang
  { $match: {
    timestamp: {
      $gte: new Date(new Date().setDate(new Date().getDate() - 7))
    }
  }
}},

```

```
// Tahap 2: Group berdasarkan sensor_id dan hitung rata-rata nilainya
{ $group: {
  _id: "$sensor_id",
  rata_rata_nilai: { $avg: "$nilai" }
}},

// Tahap 3: outputnya
{ $project: {
  _id: 0,
  sensor_id: "$_id",
  rata_rata_nilai: { $round: ["$rata_rata_nilai", 2] }, // Biar nggak kepanjangan
komanya
  total_catatan: 1
}},

// Tahap 4: Sort dari nilai rata-rata paling tinggi
{ $sort: { rata_rata_nilai: -1 } }
})
```

- **Menampilkan sensor yang memiliki nilai di atas 90 minimal 3 kali (gunakan \$group dengan \$push atau \$sum kondisional).**

```
db.data_sensor.aggregate([
// Tahap 1: Kelompokkan per sensor_id
{ $group: {
  _id: "$sensor_id",
  // Kondisional Sum: Cek tiap dokumen, kalau nilai > 90 kasih skor 1, kalau
nggak kasih 0
  total_kejadian_tinggi: {
    $sum: {
      $cond: [ { $gt: ["$nilai", 90] }, 1, 0 ]
    }
  },
  // Opsional: Kumpulin nilai-nilainya biar lu bisa liat angkanya
riwayat_nilai_tinggi: {
    $push: {
      $cond: [ { $gt: ["$nilai", 90] }, "$nilai", "$$REMOVE" ]
    }
  }
}},

// Tahap 2: Filter hasil grup (Cari yang total kejadiannya minimal 3)
{ $match: {
  total_kejadian_tinggi: { $gte: 3 }
}},
```



```
// Tahap 3: Rapihin tampilan
{ $project: {
  _id: 0,
  sensor_id: "$_id",
  total_kejadian_tinggi: 1,
  riwayat_nilai_tinggi: 1
}},

// Tahap 4: Urutin dari yang paling sering "overheat" atau nilai tinggi
{ $sort: { total_kejadian_tinggi: -1 } }
])
```

- **Menambahkan field jam dan menghitung nilai rata-rata per jam per sensor.**

```
db.data_sensor.aggregate([
// Tahap 1: Tambahkan field 'jam' dengan mengekstrak dari timestamp
{ $addFields: {
  jam: { $hour: "$timestamp" }
}},

// Tahap 2: Group berdasarkan sensor_id DAN jam
{ $group: {
  _id: {
    sensor_id: "$sensor_id",
    jam: "$jam"
  },
  rata_rata_per_jam: { $avg: "$nilai" },
  total_data: { $sum: 1 }
}},

// Tahap 3: Rapihin tampilan biar gak ribet liatnya
{ $project: {
  _id: 0,
  sensor_id: "$_id.sensor_id",
  jam: "$_id.jam",
  rata_rata: { $round: ["$rata_rata_per_jam", 2] },
  total_catatan: "$total_data"
}},

// Tahap 4: Sort biarurut per jam, terus per sensor
{ $sort: { jam: 1, sensor_id: 1 } }
])
```

- **Input Data Sensor**

```
db.sensor_info.insertMany([
  { sensor_id: "SN-019", nama_sensor: "Sensor Suhu Ruangan" },
  { sensor_id: "SN-021", nama_sensor: "Sensor Kelembaban" },
  { sensor_id: "SN-042", nama_sensor: "Sensor Tekanan Udara" },
  { sensor_id: "SN-076", nama_sensor: "Sensor Cahaya" },
  { sensor_id: "SN-026", nama_sensor: "Sensor Gas" },
  { sensor_id: "SN-033", nama_sensor: "Sensor Gerak" },
  { sensor_id: "SN-057", nama_sensor: "Sensor Suara" },
  { sensor_id: "SN-067", nama_sensor: "Sensor Getaran" },
  { sensor_id: "SN-001", nama_sensor: "Sensor Kualitas Udara" },
  { sensor_id: "SN-009", nama_sensor: "Sensor Air" }
])
```

- **Join Data Sensor**

```
db.data_sensor.aggregate([
  { $group: {
    _id: "$sensor_id",
    rata_rata: { $avg: "$nilai" }
  } },
  { $lookup: {
    from: "sensor_info",
    localField: "_id",
    foreignField: "sensor_id",
    as: "info"
  } },
  {
    $unwind: "$info"
  },
  {
    $project: {
      sensor_id: "$_id",
      nama_sensor: "$info.nama_sensor",
      rata_rata: 1,
      _id: 0
    }
  }
])
```

2. Tangkapan Layar Hasil

a. TUTORIAL TERBIMBING STUDI KASUS AGREGASI

- Latihan Tambahan

1. Trend Bulanan

```
< {
  total_target: 10575,
  total_ok: 8386,
  total_reject: 693,
  mesin: 'M09',
  bulan: 6,
  tahun: 2026,
  availability: 0.8235416666666666,
  performance: 0.7930023640661938,
  quality: 0.9236700077101002,
  OEE: 0.60322162321609
}
{
  total_target: 9872,
  total_ok: 7650,
  total_reject: 511,
  mesin: 'M01',
  bulan: 5,
  tahun: 2026,
  availability: 0.833117816091954,
  performance: 0.7749189627228525,
  quality: 0.9373851243720133,
  OEE: 0.6051747056880377
}
```

2. Mesin Terburuk

```
< {
  total_target: 10575,
  total_ok: 8386,
  total_reject: 693,
  mesin: 'M09',
  availability: 0.8235416666666666,
  performance: 0.7930023640661938,
  quality: 0.9236700077101002,
  OEE: 0.60322162321609
}
{
  total_target: 9857,
  total_ok: 7726,
  total_reject: 590,
  mesin: 'M02',
  availability: 0.8515086206896552,
  performance: 0.7838084609921883,
  quality: 0.929052429052429,
  OEE: 0.6200678577179567
}
```

3. Kinerja Harian

```
< {
  total_target: 831,
  total_ok: 537,
  total_reject: 44,
  mesin: 'M01',
  availability: 0.828125,
  performance: 0.6462093862815884,
  quality: 0.9242685025817556,
  OEE: 0.49461503181369104
}
{
  total_target: 609,
  total_ok: 444,
  total_reject: 30,
  mesin: 'M01',
  availability: 0.9354166666666667,
  performance: 0.729064039408867,
  quality: 0.9367088607594937,
  OEE: 0.6388154476107335
}
```

4. Simulasi Perbaikan

```
< {
  total_target: 288,
  total_ok: 270,
  total_reject: 40,
  mesin: 'M02',
  availability: 0.9,
  performance: 0.9375,
  quality: 0.8709677419354839,
  OEE: 0.7348790322580645
}
{
  total_target: 222,
  total_ok: 204,
  total_reject: 30,
  mesin: 'M10',
  availability: 0.9708333333333333,
  performance: 0.918918918918919,
  quality: 0.8717948717948718,
  OEE: 0.7777431277431278
}
```

b. PANDUAN TUGAS MANDIRI PERTEMUAN 5

- Menampilkan rata rata nilai per sensor_id pada 7 hari terakhir.

```
< {
  sensor_id: 'SN-019',
  rata_rata_nilai: 75.52
}
{
  sensor_id: 'SN-050',
  rata_rata_nilai: 73.61
}
{
  sensor_id: 'SN-009',
  rata_rata_nilai: 70.12
}
{
  sensor_id: 'SN-074',
  rata_rata_nilai: 67.38
}
{
  sensor_id: 'SN-065',
  rata_rata_nilai: 67.01
}
{
  sensor_id: 'SN-062',
  rata_rata_nilai: 66.65
}
{
  sensor_id: 'SN-042',
  rata_rata_nilai: 66.61
}
```

- Menampilkan sensor yang memiliki nilai di atas 90 minimal 3 kali (gunakan \$group dengan \$push atau \$sum kondisional).

```
total_kejadian_tinggi: 12,
riwayat_nilai_tinggi: [
  95.66,
  93.34,
  94.39,
  99.42,
  94.88,
  90.87,
  98.17,
  94.95,
  91,
  96.27,
  91.9,
  96.95
],
sensor_id: 'SN-050'
}
{
  total_kejadian_tinggi: 10,
  riwayat_nilai_tinggi: [
    95.27,
    93.6,
    92.81,
    94.21,
    93.69,
    98.14,
    95.08,
    96.98,
    99.42,
    94.39
  ]
}
```

- Menambahkan field jam dan menghitung nilai rata-rata per jam per sensor.

```
< {
  sensor_id: 'SN-001',
  jam: 0,
  rata_rata: 29.04,
  total_catatan: 1
}
{
  sensor_id: 'SN-002',
  jam: 0,
  rata_rata: 62.8,
  total_catatan: 5
}
{
  sensor_id: 'SN-003',
  jam: 0,
  rata_rata: 67.78,
  total_catatan: 1
}
{
  sensor_id: 'SN-004',
  jam: 0,
  rata_rata: 61.77,
  total_catatan: 3
}
```

- Input data sensor_info

```
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f85ccb007455134b3d0e67'),
    '1': ObjectId('69f85ccb007455134b3d0e68'),
    '2': ObjectId('69f85ccb007455134b3d0e69'),
    '3': ObjectId('69f85ccb007455134b3d0e6a'),
    '4': ObjectId('69f85ccb007455134b3d0e6b'),
    '5': ObjectId('69f85ccb007455134b3d0e6c'),
    '6': ObjectId('69f85ccb007455134b3d0e6d'),
    '7': ObjectId('69f85ccb007455134b3d0e6e'),
    '8': ObjectId('69f85ccb007455134b3d0e6f'),
    '9': ObjectId('69f85ccb007455134b3d0e70')
  }
}
```

- Join Data Sensor

```
< {
  rata_rata: 58.71137931034483,
  sensor_id: 'SN-009',
  nama_sensor: 'Sensor Air'
}
{
  rata_rata: 53.17188679245283,
  sensor_id: 'SN-026',
  nama_sensor: 'Sensor Gas'
}
{
  rata_rata: 53.78541666666666,
  sensor_id: 'SN-033',
  nama_sensor: 'Sensor Gerak'
}
{
  rata_rata: 54.24823529411764,
  sensor_id: 'SN-019',
  nama_sensor: 'Sensor Suhu Ruangan'
}
{
  rata_rata: 55.27571428571429,
  sensor_id: 'SN-067',
  nama_sensor: 'Sensor Getaran'
}
```

3. Penjelasan Setiap Stage

\$match: Langkah awal buat filtering data. Penting banget ditaruh di depan biar query nggak berat karena data yang nggak relevan udah dibuang duluan.

\$lookup: Fungsinya buat join antar-koleksi. Di sini gue pake buat narik data master (kayak nama sensor) ke data transaksi biar laporannya informatif.

\$unwind: Ini buat bongkar data array jadi dokumen satuan. Wajib dipake kalau kita mau analisis tiap elemen di dalam list secara terpisah.

\$group: Inti dari pengolahan data. Di sini data direkap buat nyari angka rata-rata, total produksi, atau nyari nilai tertinggi/terendah per kategori.

\$addFields: Gue pake buat nambahin field atau variabel baru hasil itung-itungan tanpa ngerubah struktur data asli di database.

\$project: Tahap buat ngerapiin hasil akhir. Di sini gue milih field mana aja yang perlu dipajang dan ganti nama field-nya biar lebih enak dibaca.

\$sort: Biar datanya nggak berantakan, gue urutin berdasarkan waktu atau besaran nilai tertentu.

\$limit: Langkah terakhir buat ngebatesin jumlah data yang keluar, jadi yang tampil cuma sampel atau data teratas yang emang dibutuhkan aja.

4. Analisis dan Kesimpulan

Analisis

Implementasi kerangka kerja *aggregation pipeline* pada MongoDB dalam praktek ini memungkinkan transformasi dataset industri yang mentah menjadi informasi manajerial yang terstruktur dan siap pakai. Alur kerja dimulai dengan optimasi melalui tahap `$match` dan `$limit` untuk memangkas beban komputasi di awal, kemudian dilanjutkan dengan penggunaan `$lookup` dan `$unwind` untuk mengintegrasikan koleksi transaksi dengan data master sensor agar identitas setiap unit produksi dapat teridentifikasi dengan jelas. Proses kalkulasi metrik utama, seperti *Overall Equipment Effectiveness* (OEE) yang mencakup aspek *Availability*, *Performance*, dan *Quality*, dilakukan secara presisi pada tahap `$group` dan `$project` dengan memanfaatkan berbagai akumulator serta operator ekspresi seperti `$dateToString`, `$hour`, dan `$cond`. Melalui pengurutan data menggunakan `$sort`, seluruh rangkaian percobaan ini berhasil menyajikan identifikasi tren bulanan, pemantauan kinerja harian, hingga deteksi anomali pada sensor secara sistematis, yang pada akhirnya membuktikan efektivitas basis data NoSQL dalam menjawab tantangan bisnis serta kebutuhan analisis data industri secara nyata.

Kesimpulan

Dalam laporan praktikum ini, gue mengimplementasikan aggregation pipeline di MongoDB buat mentransformasi data sensor mentah jadi informasi yang lebih sistematis. Alurnya dimulai dengan `$match` buat nyaring data sesuai kriteria supaya performa query tetep optimal. Setelah itu, gue gunain `$lookup` buat melakukan join data transaksi dengan data master agar identitas tiap sensor bisa teridentifikasi dengan jelas. Untuk data yang formatnya array, gue bongkar pake `$unwind` supaya tiap elemen bisa dianalisis secara individu.

Proses pengolahan utamanya ada di tahap `$group`, di mana gue melakukan rekapitulasi data buat dapetin nilai rata-rata (average) dan total produksi harian. Gue juga nambahin beberapa parameter kalkulasi tambahan lewat `$addFields`, lalu ngerapiin output-nya pake `$project` supaya field yang muncul cuma yang relevan buat laporan aja. Biar hasil akhirnya lebih rapi dan gampang dibaca, gue urutin datanya pake `$sort` dan gue batesin jumlah tampilannya lewat `$limit` sesuai kebutuhan analisis.

- Script Pembangkit Data
- Generate Inspeksi Koleksi Data Dummy

```
from pymongo import MongoClient
from datetime import datetime, timedelta
import random

# Ganti dengan NIM Anda
NIM = "225443006"
client = MongoClient('mongodb://localhost:27017/')
```



```

db = client[f'tugas5_225443006']
db.inspeksi.drop()
db.target_kualitas.drop()

mesin_list = [f"M{i:02d}" for i in range(1, 11)]
batch_ids = [f"B{str(i).zfill(4)}" for i in range(1, 2001)]
inspektor_list = ["Andi", "Budi", "Citra", "Dian", "Eka"]
jenis_cacat_options = ["gores", "retak", "bengkok", "warna", "dimensi",
"pori"]

# Generate data inspeksi (1500 dokumen)
docs = []
start_date = datetime(2026, 4, 1, 6, 0)
for i in range(1500):
    batch = random.choice(batch_ids)
    mesin = random.choice(mesin_list)
    tanggal = start_date + timedelta(minutes=random.randint(0, 60*24*90)) # 3
bulan
    shift = 1 if tanggal.hour < 14 else (2 if tanggal.hour < 22 else 3)
    jumlah = random.randint(100, 500)
    reject = int(jumlah * random.uniform(0, 0.10)) # 0-10% cacat
    jenis = random.sample(jenis_cacat_options, k=random.randint(1, 3))
    doc = {
        "batch_id": batch,
        "mesin": mesin,
        "tanggal": tanggal,
        "shift": shift,
        "inspektor": random.choice(inspektor_list),
        "jumlah_diperiksa": jumlah,
        "cacat_ditemukan": reject,
        "jenis_cacat": jenis
    }
    docs.append(doc)
    if len(docs) >= 500:
        db.inspeksi.insert_many(docs)
        docs.clear()
if docs:
    db.inspeksi.insert_many(docs)

print("Data inspeksi selesai. Total:", db.inspeksi.count_documents({}))

# Generate target kualitas (setiap batch punya target cacat maksimal 5% dari
rata-rata ukuran batch)
targets = []
for i in range(1, 2001):
    batch = f"B{str(i).zfill(4)}"
    # target cacat acak 1-15
    target_cacat = random.randint(1, 15)

```

```

        targets.append({"batch_id": batch, "target_maks_cacat": target_cacat})
    if len(targets) >= 500:
        db.target_kualitas.insert_many(targets)
        targets.clear()
if targets:
    db.target_kualitas.insert_many(targets)

print("Data target kualitas selesai. Total:",
db.target_kualitas.count_documents({}))

```

- Generate Data Sensor Data Dummy

```

import random
from datetime import datetime, timedelta
from pymongo import MongoClient

# 1. Koneksi ke MongoDB (Sesuaikan URI-nya kalau lu pake MongoDB Atlas)
try:
    client = MongoClient("mongodb://localhost:27017/")
    db = client["tugas5_225443006"] # Ganti dengan nama database yang lu mau
    collection = db["data_sensor"]
    print("Koneksi berhasil!")
except Exception as e:
    print(f"Gagal konek: {e}")

# 2. Persiapan Data Dummy
categories = ["Temperatur", "Tekanan", "Getaran", "Kelembapan", "Flow Rate"]
data_list = []
now = datetime.now()

print("Lagi proses bikin 5000 data, tunggu bentar ya wok...")

for i in range(5000):
    # Bikin waktu acak dalam 30 hari ke belakang
    random_days = random.randint(0, 30)
    random_seconds = random.randint(0, 86400)
    ts = now - timedelta(days=random_days, seconds=random_seconds)

    doc = {
        "sensor_id": f"SN-{random.randint(1, 100):03d}",
        "nilai": round(random.uniform(10.0, 100.0), 2),
        "timestamp": ts,
        "category": random.choice(categories)
    }
    data_list.append(doc)

# 3. Masukkan ke Database (Gunakan insert_many biar kenceng)
if data_list:
    result = collection.insert_many(data_list)

```

```

    print(f"Mantap wok! {len(result.inserted_ids)} data udah masuk ke koleksi 'data_sensor'.")

# Opsional: Bikin Index biar query lu nanti nggak lemot
collection.create_index([("timestamp", -1)])
collection.create_index([("sensor_id", 1)])
print("Index juga udah dibuat biar makin kenceng!")

```

- Generate Produksi Harian Data Dummy

```

from pymongo import MongoClient
from datetime import datetime, timedelta
import random

client = MongoClient('mongodb://localhost:27017/')
db = client['latihan5']
collection = db['data_sensor']
collection.drop() # Mulai bersih

mesin_list = [f"M{i:02d}" for i in range(1, 11)]
start_date = datetime(2026, 4, 1)
docs = []

for i in range(1000):
    # Acak hari dalam 90 hari
    day_offset = random.randint(0, 90)
    tanggal = start_date + timedelta(days=day_offset)
    shift = random.randint(1, 3)
    mesin = random.choice(mesin_list)
    target = random.randint(200, 500)
    actual_ok = int(target * random.uniform(0.6, 1.0)) # Kadang di bawah target
    actual_reject = int(actual_ok * random.uniform(0, 0.15)) # reject sampai 15%
    durasi_tersedia = 480 # 8 jam
    durasi_operasi = int(durasi_tersedia * random.uniform(0.7, 1.0))

    doc = {
        "mesin": mesin,
        "tanggal": tanggal,
        "shift": shift,
        "target": target,
        "actual_ok": actual_ok,
        "actual_reject": actual_reject,
        "durasi_operasi_menit": durasi_operasi,
        "durasi_tersedia_menit": durasi_tersedia
    }
    docs.append(doc)

```

```
    if len(docs) >= 500:
        collection.insert_many(docs)
        docs.clear()

if docs:
    collection.insert_many(docs)

print("Data berhasil dibuat. Total:", collection.count_documents({}))
```